

昇腾 910B 平台 ERNIE-Image 本地部署教程

面向外部用户的最小可运行部署指南 更新时间：2026-04-20 整理依据：附件《外部用户部署指导(1).txt》

0. 写在前面

这篇文档用于帮助你在 腾 910B 环境上，快速完成 ERNIE-Image 的本地推理部署。

整套流程以“先跑通第一张图”为目标，核心包括：

- 准备带 `torch_npu` 的基础镜像
- 在容器中安装 NPU 适配版 `diffusers`
- 加载本地 ERNIE-Image 模型目录
- 执行一次文生图推理，输出图片文件

本文严格以附件中提供的可执行信息为主，不额外补写未验证的服务化、训练、量化和性能压测步骤。

1. 教程亮点

你将获得什么

- 一套可直接复制执行的部署命令
- 一份最小推理脚本示例
- 一套部署完成后的验证方法
- 一份常见问题排查清单

适用场景

- 需要在 腾 910B 服务器上本地验证 ERNIE-Image
- 希望先打通 NPU 推理链路，再做进一步工程化封装
- 需要给外部用户提供一份清晰、可复用的部署说明

2. 环境要求

2.1 硬件与系统前提

建议提前准备：

- 腾 910B 服务器
- 已正常安装 Ascend 驱动的宿主机
- 宿主机可正常执行 `npu-smi`
- 已安装 Docker
- 已准备好本地 ERNIE-Image 模型目录

2.2 关键版本

附件中出现的关键版本如下：

组件	版本 / 说明
Docker 镜像	quay.io/ascend/vllm-ascend:v0.13.0
diffusers 分支	native_npu
peft	>=0.17.0
transformers	5.5.0

附件中的示例默认暴露 0,1,2,3,4,5,6,7 共 8 张 NPU。若你的实际卡数不同，请按机器配置修改 `ASCEND_RT_VISIBLE_DEVICES`。

3. 整体流程一览

整个部署流程可以概括为 5 步：

1. 拉取 腾 NPU 运行镜像
2. 启动容器并挂载驱动与工具
3. 安装 NPU 适配版 `diffusers` 和依赖
4. 准备 ERNIE-Image 模型目录
5. 执行推理脚本并检查输出图片

如果你只关心“能不能跑通”，建议直接跳到 第 8 节：最小验证流程。

4. 详细部署步骤

Step 1：拉取包含 `torch_npu` 的镜像

先拉取附件中给出的镜像：

```
docker pull quay.io/ascend/vllm-ascend:v0.13.0
```

建议：如果你所在环境访问外网较慢，可以提前确认镜像仓库连通性，避免拉取超时。

Step 2：启动容器

执行以下命令启动容器：

```
docker run -itd \
  --privileged \
  --network=host \
  --shm-size=128G \
  -v /usr/local/Ascend/driver:/usr/local/Ascend/driver \
  -v /usr/local/bin/npu-smi:/usr/local/bin/npu-smi \
  -v /usr/local/dcmi:/usr/local/dcmi \
  -e ASCEND_RT_VISIBLE_DEVICES="0,1,2,3,4,5,6,7" \
  quay.io/ascend/vllm-ascend:v0.13.0 /bin/bash
```

参数说明

参数	作用
<code>--privileged</code>	开启特权模式，方便容器访问底层设备
<code>--network=host</code>	使用宿主网络
<code>--shm-size=128G</code>	增大共享内存，降低运行时资源限制风险
<code>-v /usr/local/Ascend/driver:/usr/local/Ascend/driver</code>	挂载 Ascend 驱动
<code>-v /usr/local/bin/npu-smi:/usr/local/bin/npu-smi</code>	挂载 NPU 管理工具
<code>-v /usr/local/dcmi:/usr/local/dcmi</code>	挂载相关运行时目录
<code>ASCEND_RT_VISIBLE_DEVICES</code>	指定容器可见的 NPU 设备

启动后进入容器：

```
docker ps
docker exec -it <container_id> /bin/bash
```

Step 3 : 安装 NPU 适配版 `diffusers`

进入容器后，执行：

```
# The NPU adaptation code is awaiting open-source review.
# PR: https://github.com/huggingface/diffusers/pull/13490

git clone https://github.com/chang-zhijie/diffusers.git
cd diffusers
git checkout native_npu
pip install -e .
pip install 'peft>=0.17.0' transformers==5.5.0
```

这一部分在做什么

- 从仓库拉取 NPU 适配版 `diffusers`
- 切换到 `native_npu` 分支
- 以可编辑模式安装当前代码
- 补充 ERNIE-Image 推理所需依赖

附件中说明：该 NPU 适配代码仍在等待上游开源合入，相关 PR 已给出。

Step 4 : 准备 ERNIE-Image 模型目录

附件中的示例代码使用如下路径：

```
"/model_dir/ERNIE-Image"
```

因此，你需要提前把模型目录准备到容器可访问的位置，例如：

```
/model_dir/ERNIE-Image
```

如果你的模型不在这个路径，请在后续脚本里替换成真实路径。

Step 5：编写并执行推理脚本

在容器内新建脚本，例如 `infer_ernie_image.py`：

```
import torch
import torch_npu
from diffusers import ErnieImagePipeline
from diffusers.utils import load_image

pipe = ErnieImagePipeline.from_pretrained(
    "/model_dir/ERNIE-Image",
    torch_dtype=torch.bfloat16,
)
pipe = pipe.to("npu")
pipe.transformer.set_attention_backend("_native_npu")
generator = torch.Generator(device="npu")

prompt = "一只黑白相间的中华田园犬"
images = pipe(
    prompt=prompt,
    height=1024,
    width=1024,
    num_inference_steps=50,
    guidance_scale=5.0,
    generator=generator,
    use_pe=True,
).images

images[0].save("ernie-image-output.png")
```

执行：

```
python infer_ernie_image.py
```

运行成功后，当前目录下会生成：

```
ernie-image-output.png
```

附件原文中引入了 `load_image`，但该示例并未实际使用它。若只做首个文生图验证，可保留，也可删除。

5. 部署成功的判定标准

如果以下 3 项都满足，通常就可以认为 ERNIE-Image 已在 腾 910B 上成功跑通：

5.1 NPU 可见

```
npu-smi info
```

能正常看到设备信息，说明基础 NPU 环境正常。

5.2 模型加载成功

执行脚本时未出现以下问题：

- 模型路径不存在
- `torch_npu` 初始化失败
- `diffusers` 依赖缺失
- 设备切换到 `npu` 时报错

5.3 结果图片成功生成

确认输出文件存在：

```
ls -lh ernie-image-output.png
```

若文件存在且图片可正常打开，就说明首轮推理已经完成。

6. 常见问题与排查建议

6.1 容器里看不到 NPU

优先检查：

- 宿主机 Ascend 驱动是否安装正常
 - `/usr/local/Ascend/driver` 是否存在
 - `/usr/local/bin/npu-smi` 是否存在
 - `docker run` 的挂载参数是否遗漏
 - `ASCEND_RT_VISIBLE_DEVICES` 是否配置正确
-

6.2 模型加载失败

优先检查：

- `/model_dir/ERNIE-Image` 路径是否真实存在
 - 模型目录是否完整
 - 模型目录是否已正确挂载进容器
 - `from_pretrained()` 的路径是否与实际路径一致
-

6.3 依赖安装报错

优先检查：

- 是否成功切换到了 `native_npu` 分支
 - `pip install -e .` 是否已成功执行
 - `peft>=0.17.0` 是否已安装
 - `transformers==5.5.0` 是否已安装
 - 当前 Python 环境是否就是容器内环境
-

6.4 推理速度偏慢 / 占用偏高

附件未给出性能优化参数，因此本文只覆盖“功能可跑通”的最小路径。如果后续要继续优化，可从以下方向着手：

- 下调分辨率
- 减少 `num_inference_steps`
- 调整实际使用卡数
- 根据业务场景设计批量推理方案

7. 可直接复制的推理脚本

```
import torch
import torch_npu
from diffusers import ErnieImagePipeline

pipe = ErnieImagePipeline.from_pretrained(
    "/model_dir/ERNIE-Image",
    torch_dtype=torch.bfloat16,
)
pipe = pipe.to("npu")
pipe.transformer.set_attention_backend("_native_npu")
generator = torch.Generator(device="npu")

prompt = "一只黑白相间的中华田园犬"
images = pipe(
    prompt=prompt,
    height=1024,
    width=1024,
    num_inference_steps=50,
    guidance_scale=5.0,
    generator=generator,
    use_pe=True,
).images

images[0].save("ernie-image-output.png")
```

8. 最小验证流程

如果你只想最快完成一次部署验证，可以直接按以下顺序执行：

```
# 1) 拉镜像
docker pull quay.io/ascend/vllm-ascend:v0.13.0

# 2) 启容器
docker run -itd \
    --privileged \
    --network=host \
    --shm-size=128G \
    -v /usr/local/Ascend/driver:/usr/local/Ascend/driver \
    -v /usr/local/bin/npu-smi:/usr/local/bin/npu-smi \
    -v /usr/local/dcmi:/usr/local/dcmi \
    -e ASCEND_RT_VISIBLE_DEVICES="0,1,2,3,4,5,6,7" \
    quay.io/ascend/vllm-ascend:v0.13.0 /bin/bash

# 3) 进入容器
docker exec -it <container_id> /bin/bash

# 4) 安装依赖
git clone https://github.com/chang-zhijie/diffusers.git
cd diffusers
git checkout native_npu
pip install -e .
pip install 'peft>=0.17.0' transformers==5.5.0

# 5) 执行推理
python infer_ernie_image.py
```

9. 参考信息

- 附件原始文档: [外部用户部署指导\(1\).txt](#)
 - NPU 适配版 [diffusers](#): <<https://github.com/chang-zhijie/diffusers>>
 - 相关 PR: <<https://github.com/huggingface/diffusers/pull/13490>>
 - 参考样式来源: <<https://aistudio.baidu.com/projectdetail/10216627?channelType=0&channel=0>>
-

10. 一句话总结

如果你的宿主机驱动正常、容器能看到 NPU、模型目录路径正确，那么按照本文给出的镜像、依赖和脚本，就可以在 腾 910B 平台上完成 ERNIE-Image 的本地首轮推理验证。